

Bab 2. Struktur dan Logika Bahasa Pemrograman

Tujuan Pembelajaran

Setelah mempelajari bab ini, diharapkan mahasiswa mampu:

- Memahami perbedaan interpreter dan compiler
- Memahami struktur bahasa pemrograman
- Memahami logika berpikir bahasa pemrograman khususnya pada *flow control* (sekuensial, percabangan, dan perulangan)

Interpreter vs Compiler

Pada Bab 1, telah dijelaskan bahwa penggunaan bahasa pemrograman untuk memprogram sebuah komputer merupakan suatu yang penting, supaya bisa bekerja dalam mencari solusi permasalahan komputasi. Dalam Bab 1 telah dijelaskan pula bahwa kelebihan dari bahasa pemrograman tingkat tinggi adalah mudah dibaca dan dipelajari oleh programmer karena bentuk perintah-perintahnya mirip dengan bahasa manusia. Pertanyaan berikutnya adalah bagaimana supaya komputer bisa memahami serangkaian perintah yang ditulis dengan bahasa pemrograman tingkat tinggi tersebut? Karena dalam hal ini komputer hanya bisa memahami perintah dalam bentuk bilangan biner 0 dan 1 saja.

Komputer bisa memahami kode program yang ditulis dengan bahasa tingkat tinggi disebabkan karena adanya proses penterjemahan dari bahasa tingkat tinggi ke dalam bahasa mesin. Proses penterjemahan ini bisa dilakukan oleh interpreter atau compiler. Apa perbedaannya? Perhatikan Tabel 2.1

Tabel 2.1 Perbandingan interpreter vs compiler

Interpreter	Compiler
Menterjemahkan setiap file kode program terpisah secara langsung	Beberapa file kode program digabung menjadi satu kemudian diterjemahkan menjadi sebuah file object
Proses penterjemahannya cepat	Proses penterjemahannya agak lama
Running programnya lebih lambat	Running programnya lebih cepat
Selama proses penterjemahan, memori yang digunakan lebih kecil	Selama proses penterjemahan, memori yang digunakan lebih lama
Mudah dalam proses debugging karena dilakukan per file kode program	Agak sulit dalam proses debugging

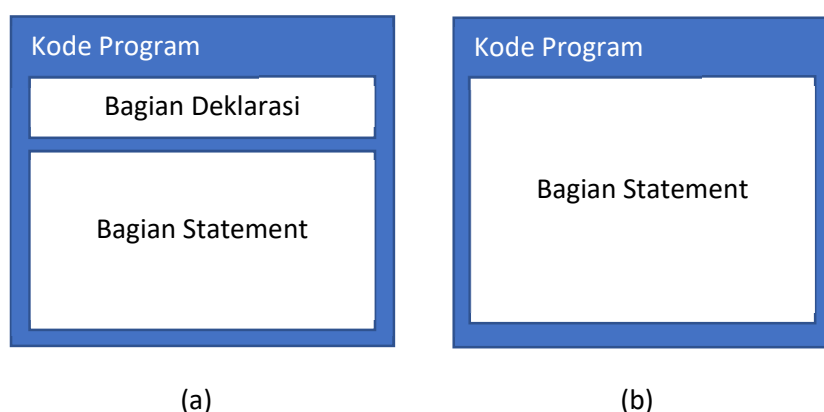
Berdasarkan Tabel 2.1 terlihat bahwa baik interpreter maupun compiler masing-masing memiliki kelebihan dan kekurangan. Adapun contoh bahasa pemrograman yang menggunakan interpreter adalah Python, PHP, dan Ruby. Sedangkan yang menggunakan compiler adalah Java, Pascal, C/C++, Delphi, Visual Basic, dll.

Struktur Bahasa Pemrograman

Seorang programmer dituntut untuk bisa membuat program yang baik. Sebuah program dikatakan baik jika memenuhi beberapa persyaratan yaitu:

- Program harus sesuai dengan spesifikasi kebutuhan.
Sebuah program yang dibuat harus bisa menangani kebutuhan yang diinginkan penggunanya, mulai dari input sampai dengan ketepatan output yang diharapkan.
- Fleksibel
Program yang baik harus fleksibel dalam pengembangan selanjutnya atau dapat dengan mudah ditambah dan dikurangi fitur-fiturnya sesuai kebutuhan.
- Tidak mengandung kesalahan
Ini merupakan syarat mutlak bahwa sebuah program harus tidak boleh mengandung bug baik yang disebabkan oleh kesalahan syntax (*syntax error*) atau kesalahan algoritma (*algorithmic error*).
- Terdokumentasi secara baik
Program yang baik juga sebaiknya memiliki dokumentasi supaya memudahkan programmer dalam melakukan pengembangan.
- Cepat dalam waktu penggunaannya
Program yang baik tidak hanya bisa memberikan output yang tepat saja. Meskipun outputnya tepat, namun seorang pengguna program juga menghendaki proses yang cepat dalam menghasilkan output.
- Efisien dalam penggunaan memori komputer
Semakin besar memori komputer yang dibutuhkan program untuk bekerja maka akan berdampak pada semakin lamanya waktu untuk memproses input menjadi output. Oleh karena itu program yang baik adalah program yang menggunakan sejumlah kecil saja resource memorinya (RAM).

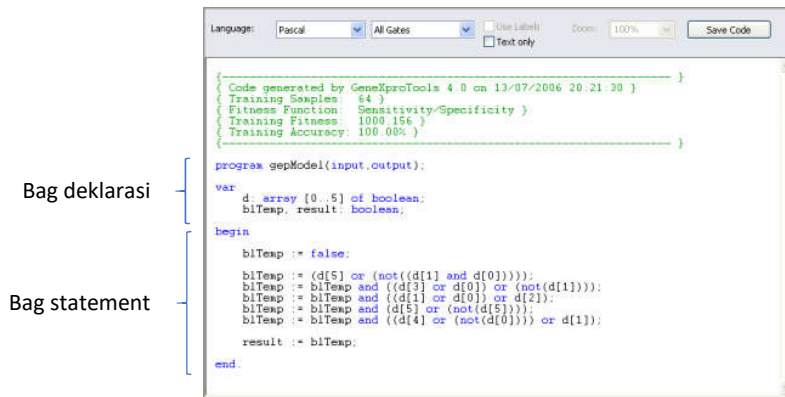
Untuk bisa membuat program dengan baik, seorang program harus memahami struktur bahasa pemrograman yang digunakannya. Secara umum, struktur bahasa pemrograman yang menggunakan struktur sebagaimana terdapat pada Gambar 2.1



Gambar 2.1 Struktur bahasa pemrograman

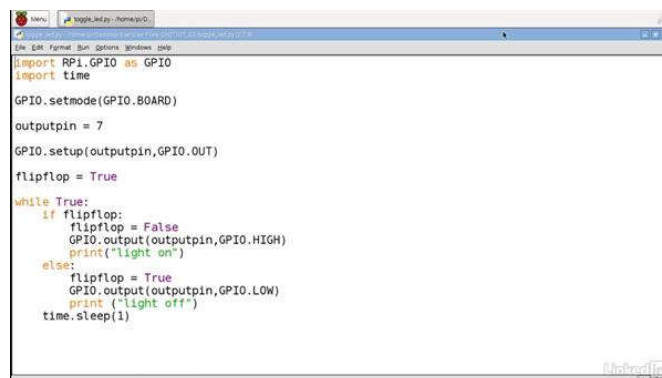
Gambar 2.1 (a) merupakan struktur bahasa pemrograman yang menggunakan compiler sebagai penterjemahnya. Struktur ini memiliki dua bagian, yaitu bagian deklarasi dan bagian statement. Bagian deklarasi biasanya terletak di atas bagian statement. Bagian ini biasanya berisi beberapa

deklarasi seperti nama program, nama variabel, tipe data, atau konstanta. Sedangkan bagian statement berisi perintah-perintah utama untuk menerima input, mengolah input, sampai dengan menampilkan outputnya. Gambar 2.2 menunjukkan contoh kode program yang dibuat dengan bahasa Pascal. Karena Pascal merupakan bahasa pemrograman yang menggunakan compiler maka strukturnya terdiri dari bagian deklarasi dan statement.



Gambar 2.2 Struktur kode program Pascal

Adapun Gambar 2.1 (b) menunjukkan struktur program yang menggunakan interpreter. Dalam hal ini tidak terdapat bagian deklarasi. Sebuah kode program yang menggunakan interpreter tidak perlu mendeklarasikan terlebih dahulu variabel apa saja yang akan digunakan beserta tipe datanya, demikian juga konstanta-konstantanya. Perhatikan contoh Gambar 2.3 yang menggambarkan struktur kode program menggunakan Python. Seluruh bagian yang ada dalam program Python berisi statement, dalam hal ini tidak ada bagian untuk mendeklarasikan nama-nama variabel yang akan digunakan beserta tipe datanya, dan juga nama-nama konstantanya.



Gambar 2.3 Struktur program Python

Di kalangan programmer, kode program yang menggunakan interpreter juga sering disebut dengan istilah *script*. Oleh karena itu ada istilah Python *script* untuk merujuk ke kode program yang dibuat dengan Python, demikian juga dengan PHP *script* dikarenakan kedua bahasa program ini menggunakan interpreter.

Logika Bahasa Pemrograman

Telah dijelaskan sebelumnya bahwa kode program yang dituliskan oleh seorang programmer berisi kumpulan instruksi yang tersusun secara teratur. Instruksi tersebut merupakan urutan langkah-langkah yang digunakan mulai dari membaca input, mengolah input, hingga menghasilkan output yang diharapkan. Kumpulan instruksi yang tersusun tersebut haruslah benar secara logika, sehingga dalam hal ini programmer tidak bisa sembarangan ketika menyusunnya. Apabila instruksi tersusun tidak benar secara logika maka akan menghasilkan output yang salah, inilah yang disebut dengan *algorithmic error* atau kesalahan algoritma.

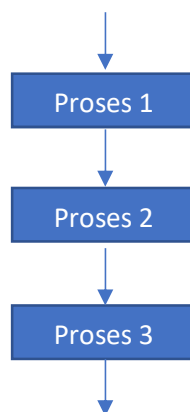
Dalam menyusun instruksi-instruksi tersebut ke dalam bahasa pemrograman, secara umum menggunakan pola berpikir yang sama dengan pola berpikir manusia secara umum ketika menyelesaikan suatu permasalahan. Dengan kata lain, pola berpikir yang digunakan dalam bahasa pemrograman mengadopsi pola berpikir manusia. Secara umum terdapat tiga jenis alur proses instruksi dalam pemrograman yaitu: sekuensial, percabangan (bersyarat), dan perulangan (loop).

Sekuensial

Instruksi dalam bahasa pemrograman secara umum tersusun secara sekuensial atau berurutan. Sebuah instruksi akan dijalankan apabila instruksi sebelumnya telah dilakukan. Jika dalam kehidupan nyata, hampir semua langkah penyelesaian suatu masalah tersusun secara sekuensial, misalnya adalah bagaimana cara memasak mie bungkus. Langkah-langkah instruksi yang bisa dilakukan untuk permasalahan ini adalah sebagai berikut:

1. Siapkan panci
2. Siapkan air sebanyak 2 gelas dan masukkan ke dalam panci
3. Rebus air dalam panci hingga mendidih
4. Masukkan mie bungkus ke dalam panci
5. Aduk-aduk selama 2-3 menit
6. Masukkan bumbu mie dan aduk-aduk mie sampai kira-kira 1 menit
7. Mie siap dihidangkan

Perhatikan langkah-langkah tersebut! Terdapat tujuh langkah untuk memasak mie bungkus sampai dengan diperoleh mie siap dihidangkan. Dalam hal ini setiap langkah (instruksi) tersusun secara sekuensial atau berurutan. Bisa jadi hasil akhir mie yang siap dihidangkan tidak bisa diperoleh apabila terdapat instruksi yang tertukar posisinya atau ada langkah yang hilang. Mekanisme proses yang terjadi dalam sekuensial seperti tampak pada Gambar 2.4.



Gambar 2.4 Proses sekuensial

Pada Gambar 2.4 dapat dipahami bahwa proses 2 tidak akan dijalankan sebelum proses 1 selesai dilakukan. Demikian juga proses 3, dalam hal ini tidak akan dijalankan sebelum proses 2 selesai dilakukan. Begitu seterusnya.

Di dalam bahasa pemrograman, konsep sekuensial juga berlaku. Setiap instruksi tersusun secara berurutan dari atas sampai bawah (top-down). Instruksi yang letaknya di atas akan dijalankan terlebih dahulu dibandingkan yang berada di bawahnya. Perhatikan contoh di Gambar 2.5

```
print("Hello World")  
print("Hello Indonesia")  
print("Hello Python")|
```

Gambar 2.5 Kode program yang tersusun secara sekuensial

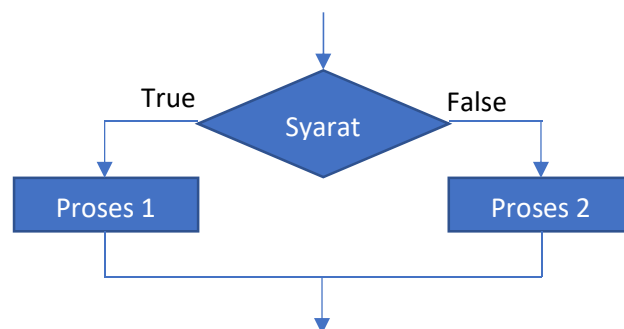
Gambar 2.5 menunjukkan ada tiga buah instruksi yang tersusun secara sekuensial di dalam sebuah program. Dilihat dari susunan urutannya, print("Hello World") akan dijalankan pertama kali, baru kemudian print("Hello Indonesia"), lalu terakhir print("Hello Python").

Percabangan (Bersyarat)

Sebuah instruksi bisa juga tidak akan dijalankan apabila suatu syarat tidak terpenuhi. Demikian pula sebaliknya, sebuah instruksi akan dijalankan ketika suatu syarat terpenuhi. Contoh dalam kehidupan sehari-hari misalnya masih terkait pada masalah merebus mie kuah.

1. Siapkan panci
2. Siapkan air sebanyak 2 gelas dan masukkan ke dalam panci
3. Rebus air dalam panci
4. Jika air sudah mendidih, maka masukkan mie bungkus ke dalam panci
Jika belum, maka tunggu sampai mendidih
5. Aduk-aduk selama 2-3 menit
6. Masukkan bumbu mie dan aduk-aduk mie sampai kira-kira 1 menit
7. Mie siap dihidangkan

Perhatikan pada langkah nomor 4 di atas. Langkah nomor 4 tersebut terdapat sebuah instruksi yang akan dijalankan yaitu 'memasukkan mie ke dalam panci' apabila suatu syarat terpenuhi yaitu 'jika air sudah mendidih'. Artinya bahwa instruksi 'memasukkan mie ke dalam panci' tidak akan dilakukan apabila air belum mendidih, dalam hal ini instruksi yang akan dijalankan adalah 'tunggu sampai mendidih'.



Gambar 2.6 Diagram alir mekanisme bersyarat

Logika bersyarat seperti ini, apabila dinyatakan dalam bentuk diagram alir seperti pada Gambar 2.6. Pada diagram tersebut tampak bahwa proses 1 akan dijalankan apabila syarat bernilai TRUE (syarat terpenuhi), sedangkan jika belum terpenuhi (FALSE) maka proses 2 nya yang akan dijalankan.

Gambar 2.7 menunjukkan contoh kode program dalam bahasa Python yang menggunakan bentuk percabangan.

```
bil = int(input("Masukkan sebuah bilangan : "))
if (bil >= 0):
    print("Anda memasukkan bilangan non negatif")
else:
    print("Anda memasukkan bilangan negatif")
```

Gambar 2.7 Contoh penggunaan percabangan dalam Python

Perulangan (Loop)

Dalam keseharian sering dijumpai suatu proses yang dilakukan secara berulang-ulang. Sebagai contoh dalam proses membuat kopi manis berikut ini:

1. Siapkan cangkir atau gelas
2. Masukkan 1-2 sendok teh bubuk kopi
3. Tuangkan air panas ke dalam cangkir atau gelas
4. Masukkan gula ke dalam cangkir atau gelas
5. Aduk-aduk
6. Cicipi kopinya
7. Jika masih belum manis, maka ulangi langkah 4 sampai dengan 6
Jika sudah manis, maka lanjutkan langkah 8
8. Kopi manis siap dinikmati

Apabila diperhatikan, maka pada proses membuat kopi tersebut terdapat beberapa instruksi yang diulang-ulang yaitu langkah nomor 4 hingga 7. Instruksi tersebut diulang sampai dengan terpenuhi syarat tertentu, dalam hal ini syaratnya yaitu sampai dengan kopi sudah terasa manis. Melalui syarat itulah proses perulangannya berhenti.

Di dalam konsep pemrograman, terdapat beberapa jenis perulangan. Sebelum membahas lebih lanjut tentang hal ini, perhatikan tiga contoh bentuk perulangan dalam kalimat sehari-hari berikut ini:

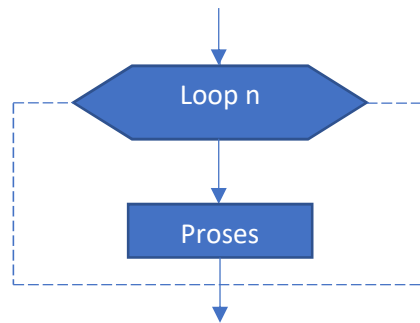
- a. Saya menuangkan segelas air ke dalam ember sebanyak 10 kali
- b. Selama ember belum penuh, saya akan tetap menuangkan segelas air ke dalamnya
- c. Saya terus menuangkan segelas air ke dalam ember hingga embernnya penuh

Ketiga bentuk perulangan di atas ke semuanya mengandung makna perulangan, dengan kata lain ada proses yang diulang-ulang di dalamnya yaitu menuangkan segelas air ke dalam ember. Kalimat (a) bermakna bahwa proses penuangan segelas air ke dalam ember dilakukan sebanyak 10 kali. Kalimat ini sudah jelas menerangkan banyaknya perulangan yang dilakukan. Secara diagram, proses yang terjadi dalam bentuk perulangan ini digambarkan pada Gambar 2.8.

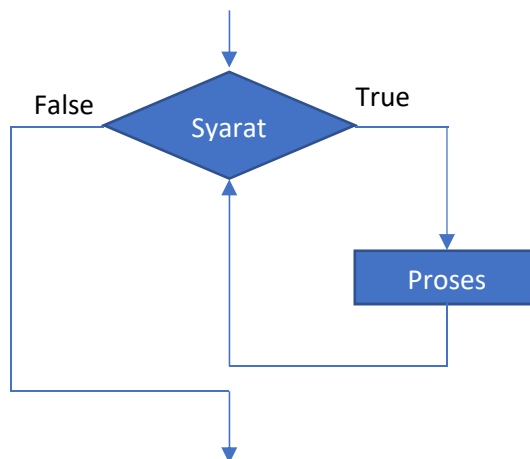
Sedangkan kalimat (b) dan (c), banyaknya perulangan yang dilakukan tidak diketahui. Dalam hal ini yang diketahui hanyalah syarat kapan perulangan tersebut berhenti dilakukan. Meskipun kedua kalimat ini mirip, namun ada perbedaan pada letak syaratnya. Pada kalimat (b) letak syarat berada di awal kalimat yaitu selama ember belum penuh. Jika syarat ini masih terpenuhi (bernilai TRUE) maka proses penuangan segelas air ini akan tetap terus dilakukan. Selanjutnya jika syarat sudah tidak

terpenuhi (bernilai FALSE), maka perulangannya berhenti. Oleh karena itu makna syarat dalam perulangan bentuk ini adalah syarat untuk tetap dilakukannya perulangan.

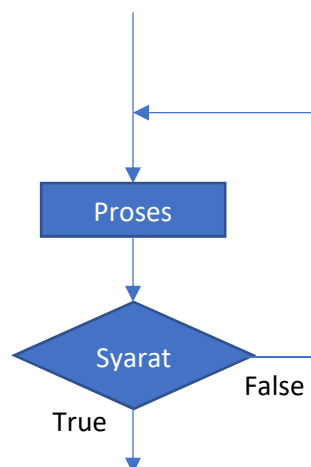
Sedangkan pada kalimat (c), letak syarat ada di akhir kalimat. Syarat yang ada di sini menunjukkan kapan perulangan berhenti, sehingga apabila syarat ini belum memenuhi (bernilai FALSE) maka perulangan akan dilakukan. Namun, jika syarat berhenti ini sudah memenuhi (bernilai TRUE) maka perulangan akan berhenti. Secara diagram, bentuk perulangan seperti kalimat (b) dijelaskan pada Gambar 2.9. Adapun untuk bentuk perulangan pada kalimat (c) dijelaskan pada Gambar 2.10.



Gambar 2.8 Diagram perulangan sebanyak n kali



Gambar 2.9 Diagram perulangan dengan syarat terletak di awal



Gambar 2.10 Diagram perulangan dengan syarat terletak di akhir

Beberapa bentuk perulangan yang telah dijelaskan dapat diterapkan dalam bahasa pemrograman. Gambar 2.11 menunjukkan contoh penggunaan masing-masing bentuk perulangan dalam bahasa Python. Gambar 2.11 (a) menunjukkan perulangan yang sudah diketahui banyaknya perulangannya. Adapun bentuk perulangan yang ada dalam Gambar 2.11 (b) menggunakan bentuk perulangan kedua yaitu adanya syarat yang terletak di awal perulangan ($i < 5$). Sedangkan Gambar 2.11 (c) adalah contoh bentuk perulangan yang ke tiga yaitu letak syarat ada di bawah.

```
for i in range(5):  
    print("Hello world")
```

(a)

```
i = 1  
while (i < 5):  
    print("Hello world")  
    i = i + 1
```

(b)

```
i = 1  
while (True):  
    print("Hello world")  
    i = i + 1  
    if (i > 4):  
        break
```

(c)

Gambar 2.11 Contoh bentuk-bentuk perulangan dalam program Python